

These 30 SQL Join questions and their corresponding answers cover a wide range of scenarios and concepts related to SQL Joins. Practicing these questions will enhance your understanding of SQL joins, allowing you to confidently handle interview questions and real-world data manipulation challenges.

Question 1

What is the difference between INNER JOIN and LEFT JOIN in SQL?

INNER JOIN returns only the rows with matching values in both tables, while LEFT JOIN returns all rows from the left table and the matching rows from the right table. If there are no matches in the right table for a left join, NULL values are returned.

Question 2

How do you perform a simple INNER JOIN between two tables in SQL?

You can use the **JOIN** keyword or the **INNER JOIN** keyword followed by the **ON** clause specifying the join condition.

For example:

```
SELECT * FROM table1
JOIN table2
ON table1.column_name = table2.column_name;
```

Question 3

Explain the usage of RIGHT JOIN with an example.

RIGHT JOIN returns all rows from the right table and the matching rows from the left table. If there are no matches in the left table, NULL values are returned.

For example:

```
SELECT * FROM employees
RIGHT JOIN departments ON
employees.department_id =
departments.department_id;
```

Question 4

How can you perform a **FULL JOIN** to retrieve all rows from two tables, even when there are no matches?

You can use the **FULL JOIN** or combine the **LEFT JOIN** and **RIGHT JOIN** using the **UNION** operator.

For example:

```
SELECT * FROM table1
FULL JOIN table2
ON table1.column_name = table2.column_name;
```

Question 5

What is a **SELF JOIN**, and in what scenarios is it useful?

A **SELF JOIN** is a join where a table is joined with itself. It is useful in scenarios where you need to compare records within the same table, such as finding employees who share the same manager.

Question 6

How do you perform a **SELF JOIN** in SQL?

You can use table aliases to distinguish between the two instances of the same table.
For example:

```
SELECT e1.employee_id, e1.name AS
employee_name, e2.name AS manager_name FROM
employees e1
JOIN employees e2
ON e1.manager_id = e2.employee_id;
```

Question 7

What is the result of a **CROSS JOIN**?

A CROSS JOIN (also known as a Cartesian Join) returns the Cartesian product of both tables, combining every row from the first table with every row from the second table.

Question 8

How do you perform a CROSS JOIN in SQL?

You can use the **CROSS JOIN** keyword to perform a cross join.

For example:

```
SELECT * FROM table1 CROSS JOIN table2;
```

Question 9

Explain the concept of Non-Equi Join.

A Non-Equi Join uses join conditions other than equality. For example, using greater than or less than operators to compare values between tables.

Question 10

How do you handle NULL values during joins?

To handle NULL values during joins, you can use the **IS NULL** or **IS NOT NULL** condition in the **WHERE** clause or coalesce NULL values using the **COALESCE** function.

Question 11

What is the difference between an INNER JOIN and an equi-join?

An INNER JOIN is a type of equi-join, which means the join condition uses the equality operator (=) to match values between tables.

Question 12

How can you retrieve records from the left table that do not have

corresponding matches in the right table?

You can use a LEFT JOIN and specify a condition in the **WHERE** clause to filter out rows where the right table's columns are NULL.

For example:

```
SELECT * FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name
WHERE table2.column_name IS NULL;
```

Question 13

What is the difference between a LEFT JOIN and a RIGHT JOIN?

The difference lies in the order of tables in the join. LEFT JOIN returns all rows from the left table and matching rows from the right table, while RIGHT JOIN returns all rows from the right table and matching rows from the left table.

Question 14

How do you perform a multi-table join in SQL?

You can perform a multi-table join by chaining multiple JOIN clauses together or using parentheses to group joins.

For example:

```
SELECT * FROM table1 JOIN table2 ON
table1.column_name = table2.column_name
JOIN table3 ON table2.column_name =
table3.column_name; -- OR SELECT * FROM
(table1 JOIN table2 ON
table1.column_name = table2.column_name) JOIN table3
ON table2.column_name = table3.column_name;
```

Question 15

How can you find records with no matching entries in either of the two joined tables?

You can use a FULL JOIN and specify a condition in the **WHERE** clause to filter out rows where either the left or right table's columns are NULL.

For example:

```
SELECT * FROM table1 FULL JOIN table2
ON table1.column_name =
table2.column_name WHERE
table1.column_name IS NULL OR
table2.column_name IS NULL;
```

Question 16

When should you use an INNER JOIN over a LEFT JOIN or RIGHT JOIN?

Use an INNER JOIN when you want to retrieve only the rows that have matching values in both tables. Use LEFT JOIN when you want all rows from the left table and matching rows from the right table. Use RIGHT JOIN when you want all rows from the right table and matching rows from the left table.

Question 17

How can you find records with matching values in one table but not in another?

You can use a LEFT JOIN and specify a condition in the **WHERE** clause to filter out rows where the right table's columns are NULL.

For example:

```
SELECT * FROM table1 LEFT JOIN table2
ON table1.column_name =
table2.column_name
WHERE table2.column_name IS NULL;
```

Question 18

What are the key differences between a FULL JOIN and a UNION?

FULL JOIN combines data from both tables, retaining non-matching rows with NULLs, while UNION combines the result sets of two or more SELECT queries, removing duplicate rows.

Question 19

How do you combine results from multiple queries without using a join?

You can use the UNION operator to combine results from multiple SELECT queries.

For example:

```
SELECT column_name FROM table1
UNION
SELECT column_name FROM table2;
```

Question 20

What is the difference between a self join and a regular join?

A self join is a join where a table is joined with itself, while a regular join is a join between two different tables. Self joins are useful when comparing records within the same table.

Question 21

How can you find the top N employees based on their salary using joins?

You can use the **ORDER BY** clause in combination with the **LIMIT** or **TOP** clause (depending on the database system) to find the top N employees based on their salary.

For example, in MySQL or PostgreSQL:

```
SELECT employee_id, name, salary
FROM employees
ORDER BY salary DESC LIMIT N;
```

In SQL Server:

```
SELECT TOP N employee_id, name, salary
FROM employees
ORDER BY salary DESC;
```

Question 22

How can you find employees who have the same manager using a self join?

You can use a self join on the **manager_id** column to find employees who share the same manager.

For example:

```
SELECT e1.employee_id, e1.name AS
employee_name, e2.name AS manager_name FROM
employees e1
JOIN employees e2 ON e1.manager_id = e2.manager_id
WHERE e1.employee_id <> e2.employee_id;
```

Question 23

What is the purpose of using table aliases in SQL joins?

Table aliases provide a shorthand notation for table names, improving query readability and reducing the amount of typing. They are especially useful when dealing with self joins or joining multiple tables.

Question 24

How can you join three or more tables in a single query?

You can use multiple JOIN clauses to join three or more tables together.

For example:

```
SELECT * FROM table1
JOIN table2 ON table1.column_name =
table2.column_name JOIN table3 ON
table2.column_name = table3.column_name;
```

Question 25

Explain how you can perform a LEFT JOIN between two tables and keep only the rows that do not match.

To perform a LEFT JOIN and keep only the rows with no match, you can specify a condition in the **WHERE** clause that checks for NULL values in the right table's columns.

For example:

```
SELECT * FROM table1
LEFT JOIN table2 ON table1.column_name =
table2.column_name WHERE table2.column_name IS
NULL;
```

Question 26

Can you use a JOIN without specifying a join condition? If yes, what will be the result?

Yes, you can use a CROSS JOIN (Cartesian Join) without specifying a join condition. The result will be a combination of every row from the first table with every row from the second table, resulting in a large result set.

Question 27

What is the benefit of using an equi-join over a non-equi join?

Equi-joins are more straightforward and easier to read than non-equi joins. Additionally, equi-joins are often more efficient and can take advantage of indexes.

Question 28

How do you perform a LEFT JOIN between two tables and include all rows from the left table, even if there are multiple matches in the right table?

To include all rows from the left table, even with multiple matches in the right table, you can use the **GROUP BY** clause along with aggregate functions like **GROUP_CONCAT()** (MySQL) or **STRING_AGG()** (PostgreSQL).

For example:

```
SELECT t1.column_name, GROUP_CONCAT(t2.column_name)
AS matching_values
FROM table1 t1 LEFT JOIN table2 t2 ON
t1.column_name = t2.column_name
GROUP BY t1.column_name;
```

Question 29

How do you perform an anti-join (retrieve records that exist in one table but not in another)?

You can perform an anti-join by using a LEFT JOIN and checking for NULL values in the right table's columns in the **WHERE** clause.

For example:

```
SELECT * FROM table1 LEFT JOIN table2 ON
table1.column_name = table2.column_name
WHERE table2.column_name IS NULL;
```

Question 30

How can you find the average salary of employees in each department using SQL joins?

You can use a combination of the **AVG()** aggregate function and a JOIN to calculate the average salary of employees in each department.

For example:

```
SELECT d.department_id, d.department_name,
AVG(e.salary) AS avg_salary
FROM departments d
```

```
JOIN employees e ON d.department_id =  
e.department_id GROUP BY d.department_id,  
d.department_name;
```